

第 2 章：DP (Dynamic Programming, 动态规划)、Policy Evaluation 与 Value Iteration

2.1 本章符号说明

符号/缩写	English	中文	含义
DP	Dynamic Programming	动态规划	已知环境模型时，通过 Bellman backup 求解价值和策略。
$P(s' s, a)$	Transition Probability	状态转移概率	从 s 执行 a 后到 s' 的概率。
$R(s, a, s')$	Reward Function	奖励函数	从 s 执行 a 到 s' 时的奖励。
π / π_{new}	Policy / New Policy	策略 / 新策略	当前被评估或改进的策略。
$V_{\pi}(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的长期价值。
$Q_{\pi}(s, a)$	Action-value Function	动作价值函数	策略 π 下状态-动作对 (s, a) 的长期价值。
$V_k(s)$	Value Estimate at iteration k	第 k 次迭代的价值估计	value iteration / policy evaluation 中的中间估计。
γ	Discount Factor	折扣因子	控制未来 reward 权重。
GPI	Generalized Policy Iteration	广义策略迭代	policy evaluation 和 policy improvement 交替推进的通用框架。
MDP	Markov Decision Process	马尔可夫决策过程	本章 DP 要求解的对象。
RL	Reinforcement Learning	强化学习	DP 是 RL 的理论原型之一。
TD	Temporal Difference	时序差分	后续用采样近似 Bellman backup 的方法。
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似 Q-learning 的算法。
SARSA	State-Action-Reward-State-Action	状态-动作-奖励-状态-动作	第 3 章会讲的采样式 TD control 算法。

2.2 本章目标

本章学习在已知环境模型 P 和 R 的情况下，如何通过动态规划求解 MDP。

你需要掌握：

- Policy Evaluation。
- Policy Improvement。
- Policy Iteration。
- Value Iteration。
- Generalized Policy Iteration。
- DP 方法的局限性。

2.3 本章主线

本章回答一个理想化问题：如果 $P(s'|s, a)$ 和 $R(s, a, s')$ 都已知，Bellman equation 能不能直接求解？答案是可以。Policy Evaluation 负责“评估固定策略”，Policy Improvement 负责“按价值改进策略”，两者交替就是 GPI。第 3 章会把这里的精确 Bellman backup 换成采样 backup。

2.4 动态规划适用前提

动态规划类方法通常假设环境模型已知：

$$P(s' | s, a), R(s, a, s')$$

也就是说，agent 知道每个动作会以什么概率转移到什么状态，并得到什么 reward。

这在真实问题中经常不成立，但 DP 仍然重要，因为它给出了强化学习算法的理论原型。后面的 TD、Q-learning、DQN 都可以看成是在没有完整模型时，用采样近似 Bellman backup。

2.5 Policy Iteration 路线：先评估再改进

2.5.1 Policy Evaluation

Policy Evaluation 的目标：给定一个策略 π ，计算它的价值函数 $V_\pi(s)$ 。

Bellman expectation equation：

$$\begin{aligned} V_\pi(s) &= \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma V_\pi(s')] \end{aligned}$$

迭代更新：

$$\begin{aligned} V_{k+1}(s) &= \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma V_k(s')] \end{aligned}$$

直觉：

1. 初始化所有状态价值。
2. 使用 Bellman expectation backup 更新每个状态。
3. 重复直到价值函数收敛。

伪代码：

```
initialize V(s) arbitrarily
repeat:
    delta = 0
    for each state s:
        v_old = V(s)
        V(s) = sum_a pi(a|s) sum_s' P(s'|s,a) [r + gamma V(s')]
        delta = max(delta, |v_old - V(s)|)
until delta < threshold
```

2.5.2 Policy Improvement

有了 $V_{\pi}(s)$ 之后，可以通过贪心方式改进策略：

$$\pi_{new}(s) = \arg \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_{\pi}(s')]$$

直觉：

如果在某个状态下，选择动作 a 后的期望回报比当前策略更好，就应该把策略改成选择 a 。

Policy Improvement Theorem：

如果对所有状态都有：

$$Q_{\pi}(s, \pi_{new}(s)) \geq V_{\pi}(s)$$

那么新策略不差于旧策略：

$$V_{\pi_{new}}(s) \geq V_{\pi}(s)$$

2.5.3 Policy Iteration

Policy Iteration 在两个步骤之间交替：

1. Policy Evaluation：评估当前策略。
2. Policy Improvement：根据价值函数改进策略。

流程：

```
initialize pi randomly
repeat:
    V_pi = policy_evaluation(pi)
    pi_new = greedy_policy(V_pi)
    if pi_new == pi:
        stop
    pi = pi_new
```

它最终会收敛到最优策略 π^* 。

面试表达：

Policy Iteration 是 evaluation 和 improvement 的交替过程。先精确或近似评估当前策略，再对价值函数做贪心改进。这个过程可以看成广义策略迭代的一种具体形式。

2.6 Value Iteration 路线：直接做最优 backup

2.6.1 Value Iteration

Value Iteration 直接使用 Bellman optimality backup：

$$\begin{aligned} V_{k+1}(s) &= \max_a \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma V_k(s')] \end{aligned}$$

和 Policy Iteration 的区别：

- Policy Iteration 每轮先完整或近似评估一个策略，再改进策略。
- Value Iteration 把 evaluation 和 improvement 合在一次 backup 中。
- Value Iteration 每次更新都直接对动作取 max。

最后从 V^* 导出策略：

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')]$$

伪代码：

```
initialize V(s) arbitrarily
repeat:
    delta = 0
    for each state s:
        v_old = V(s)
        V(s) = max_a sum_{s'} P(s'|s,a) [r + gamma V(s')]
        delta = max(delta, |v_old - V(s)|)
until delta < threshold
derive pi from V
```

2.7 GPI 框架与 DP 局限

2.7.1 Generalized Policy Iteration

Generalized Policy Iteration, 简称 GPI, 指的是 policy evaluation 和 policy improvement 相互作用的一大类方法。

核心思想：

- evaluation 让 value function 更接近当前策略的真实价值。
- improvement 让 policy 根据当前 value function 变得更贪心。
- 两者交替推进，最终趋向最优策略和最优价值函数。

很多 RL 算法都可以放进这个框架：

- Policy Iteration: 完整 evaluation + 贪心 improvement。
- Value Iteration: 每一步都做截断 evaluation + improvement。
- SARSA: 用同一套采样策略同时做 evaluation 和 improvement; 第 3 章会把这类方法称为 on-policy。
- Q-learning: 用采样逼近 optimal Bellman backup。
- Actor-Critic: critic 做 evaluation, actor 做 improvement。

2.7.2 DP 的局限性

动态规划方法有两个主要限制：

1. 需要已知完整环境模型 P 和 R 。
2. 状态空间和动作空间大时计算成本很高。

这也是为什么后续要学习 model-free RL：

- Monte Carlo 不需要环境模型，但要等 episode 结束。
- TD 不需要模型，也不需要等 episode 结束。
- Deep RL 用神经网络处理大规模状态空间。

2.8 本章例子：小型 GridWorld 上的 DP

假设一个 3x3 GridWorld：

- 右下角是终点，reward 为 +10。
- 撞墙留在原地，reward 为 -1。
- 普通移动 reward 为 -0.1。
- 每个动作都是确定性的。

2.8.1 Policy Evaluation 例子

如果当前策略是在每个格子随机选择上下左右，那么 policy evaluation 做的是：

固定这个随机策略，计算每个格子的长期价值。

直觉结果：

- 靠近终点的格子价值更高。
- 靠近墙角但远离终点的格子价值较低。
- 如果随机策略经常撞墙，撞墙附近的价值会被拉低。

这时算法只是在回答：“如果我一直按这个随机策略走，每个格子平均值多少钱？”

2.8.2 Policy Improvement 例子

有了 $V_{\pi}(s)$ 后，在某个格子比较四个动作：

向上：下一格价值 1.2
向下：下一格价值 4.8
向左：撞墙，价值 -1.0
向右：下一格价值 3.5

policy improvement 会选择“向下”，因为它的一步 reward 加未来价值最大。

2.8.3 Value Iteration 例子

value iteration 不等完整 policy evaluation 收敛，而是每轮直接做：

每个格子都看四个动作，选 Bellman backup 最大的那个。

所以它更像“边估值，边贪心改策略”。在小 GridWorld 上，这会逐渐形成指向终点的箭头场。

面试表达：

DP 的例子一定要强调“已知模型”。GridWorld 里我知道每个动作会到哪个格子，所以能对下一状态做 Bellman backup。到了真实环境里不知道转移模型，就要转向 MC/TD。

2.9 本章面试高频问题

2.9.1 Policy Evaluation 是什么？

给定策略 π ，根据 Bellman expectation equation 计算该策略下每个状态的价值 $V_{\pi}(s)$ 。

2.9.2 Policy Iteration 和 Value Iteration 的区别？

Policy Iteration 交替进行策略评估和策略改进。Value Iteration 直接使用 Bellman optimality backup，把评估和改进合并到一次 max backup 中。

2.9.3 为什么 DP 方法在真实问题中受限？

因为它通常要求环境模型已知，即知道完整的状态转移概率和奖励函数。而真实环境中模型往往未知，且状态动作空间可能极大。

2.9.4 Value Iteration 为什么能得到最优策略？

它反复应用 Bellman optimality operator。在折扣 MDP 中，该 operator 是 contraction mapping，因此价值函数会收敛到唯一的最优价值函数 V^* ，再由 V^* 导出最优策略。

2.9.5 GPI 是什么？

GPI 是 policy evaluation 和 policy improvement 交替推进的通用思想。很多强化学习算法都可以看成不同形式的 GPI。

2.10 本章练习

1. 手写 Policy Evaluation 的 Bellman update。
2. 对比 Policy Iteration 和 Value Iteration 的伪代码。
3. 解释为什么 Value Iteration 中可以不完整评估一个策略。
4. 思考：如果状态空间有一亿个状态，表格型 DP 为什么不可行？

▼ 参考答案（点击展开）

1. **Policy Evaluation 更新**： $V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_k(s')]$ 。反复对所有状态执行该 Bellman expectation backup，直到变化足够小。
2. **两种算法的伪代码差异**：Policy Iteration 的外层循环是“把当前 policy 评估到近似收敛，再做一次 greedy improvement”；Value Iteration 每轮只做一次 optimality backup： $V_{k+1}(s) = \max_a \sum_{s'} P(s'|s,a) [R + \gamma V_k(s')]$ ，收敛后再从 V^* 提取 greedy policy。
3. **为什么不必完整评估**：策略评估和策略改进可以交错进行。Bellman optimality operator 是 contraction；只要持续做 optimality backup，价值估计就会向 V^* 收敛。Value Iteration 等价于把“有限次评估”和“立即改进”合并为一步。
4. **一亿状态为什么不可行**：至少要存储一亿个 value；每次 sweep 还要遍历每个 state、action 和可能的 next state，并要求完整的 P 、 R 模型。内存、计算量和模型获取成本都不可接受，而且无法对未枚举状态泛化，因此需要采样和函数逼近。